

234. 回文链表

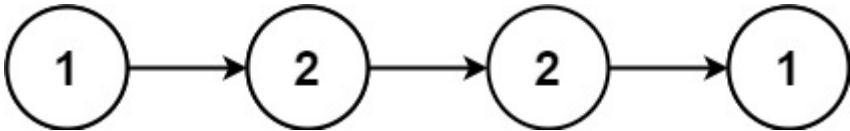
回文链表

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Easy (56.37%)	2055	0	-	-	0

- Tags
- Companies

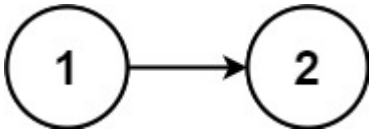
给你一个单链表的头节点 `head`，请你判断该链表是否为回文链表。如果是，返回 `true`；否则，返回 `false`。

示例 1:



```
1 输入: head = [1,2,2,1]
2 输出: true
```

示例 2:



```
1 输入: head = [1,2]
2 输出: false
```

提示:

- 链表中节点数目在范围 `[1, 105]` 内
- `0 <= Node.val <= 9`

进阶: 你能否用 `O(n)` 时间复杂度和 `O(1)` 空间复杂度解决此题?

[Discussion](#) | [Solution](#)

2.方法

LeetCode 234: 回文链表（详细解析）

1. 问题理解

目标: 判断一个单链表是否为回文链表。

回文定义： 正着读和倒着读都一样。例如"上海自来水来自海上"是回文。

输入： 单链表的头节点 `head`

输出： 如果链表是回文的，返回 `true`；否则返回 `false`

示例：

- 输入： `[1,2,2,1]`
- 输出： `true` (从前到后读是 $1 \rightarrow 2 \rightarrow 2 \rightarrow 1$ ，从后到前读也是 $1 \rightarrow 2 \rightarrow 2 \rightarrow 1$)

2. 普通人的思路（及遇到的困难）

思路一：转换为数组后判断

想法： 把链表的值都放到一个数组里，然后判断数组是否是回文的。

步骤：

1. 遍历链表，将每个节点的值存入数组
2. 使用双指针法判断数组是否为回文（一个指针从头开始，一个从尾开始，向中间移动并比较）

复杂度：

- 时间复杂度： $O(n)$
- 空间复杂度： $O(n)$ ，需要额外空间存储所有节点值

缺点： 需要额外的 $O(n)$ 空间，不是最优解。

3. 更优解法（空间复杂度 $O(1)$ 的原地判断）

核心思想： 将链表对半分，反转后半部分，然后比较前半部分和反转后的后半部分是否相同。

完整步骤：

1. **找到链表 midpoint：** 使用快慢指针技巧
 - 慢指针每次走一步，快指针每次走两步
 - 当快指针到达末尾时，慢指针正好在中点位置
2. **反转后半部分链表：** 使用之前学过的链表反转技巧
 - 从中点之后的节点开始反转
3. **比较两半链表：** 分别遍历前半部分和反转后的后半部分
 - 如果有任何值不同，则不是回文
4. **恢复链表（可选）：** 将反转的后半部分再反转回来，恢复原链表结构

4. 示例演示

以链表 `[1,2,2,1]` 为例：

1. **找到中点：**

```

1  初始:
2  slow = 1 -> 2 -> 2 -> 1
3  fast = 1 -> 2 -> 2 -> 1
4
5  第一次移动后:
6  slow = 2 -> 2 -> 1
7  fast = 2 -> 1
8
9  第二次移动后:
10 slow = 2 -> 1
11 fast = null (到达末尾)
12

```

中点是第一个值为2的节点

2. 反转后半部分:

```

1  反转前: 1 -> 2 -> 2 -> 1
2  反转后: 1 -> 2 <- 2 <- 1
3           |
4           null
5

```

前半部分: 1 -> 2

反转后的后半部分: 1 -> 2

3. 比较两半:

```

1  前半部分的第一个节点(1) == 反转后半部分的第一个节点(1) ✓
2  前半部分的第二个节点(2) == 反转后半部分的第二个节点(2) ✓
3

```

所有节点值都匹配, 是回文链表!

5. 代码实现

```

1  /**
2   * Definition for singly-linked list.
3   * public class ListNode {
4   *     int val;
5   *     ListNode next;
6   *     ListNode() {}
7   *     ListNode(int val) { this.val = val; }
8   *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
9   * }
10 */
11 class Solution {
12     /**
13      * 判断链表是否为回文链表
14      * 方法: 找中点 + 反转后半部分 + 比较
15      */
16     public boolean isPalindrome(ListNode head) {
17         // 处理边界情况

```

```

18     if (head == null || head.next == null) {
19         return true; // 空链表或只有一个节点的链表都是回文
20     }
21
22     // 1. 使用快慢指针找到链表中点
23     ListNode slow = head;
24     ListNode fast = head;
25
26     while (fast != null && fast.next != null) {
27         slow = slow.next; // 慢指针每次前进一步
28         fast = fast.next.next; // 快指针每次前进两步
29     }
30
31     // 此时 slow 指向中点（奇数长度）或中点后一个节点（偶数长度）
32
33     // 2. 反转后半部分链表
34     ListNode secondHalfHead = reverseList(slow);
35     ListNode firstHalfHead = head;
36
37     // 3. 比较前半部分和反转后的后半部分
38     ListNode p1 = firstHalfHead;
39     ListNode p2 = secondHalfHead;
40     boolean result = true;
41
42     while (result && p2 != null) {
43         if (p1.val != p2.val) {
44             result = false;
45         }
46         p1 = p1.next;
47         p2 = p2.next;
48     }
49
50     // 4. (可选) 恢复链表原结构 - 再次反转后半部分
51     // slow = reverseList(secondHalfHead);
52
53     return result;
54 }
55
56 /**
57  * 反转链表（与LeetCode 206相同）
58  */
59 private ListNode reverseList(ListNode head) {
60     ListNode prev = null;
61     ListNode curr = head;
62
63     while (curr != null) {
64         ListNode next = curr.next;
65         curr.next = prev;
66         prev = curr;
67         curr = next;
68     }
69
70     return prev;
71 }
72 }
73

```

6. 代码解析

找中点部分：

- 使用经典的快慢指针技巧找到链表中点
- 快指针每次移动两步，慢指针每次移动一步
- 当快指针到达末尾时，慢指针指向中点（对于奇数长度）或中点后一个节点（对于偶数长度）

反转链表部分：

- 使用之前学过的迭代法反转链表
- 从中点开始反转后半部分链表

比较部分：

- 分别从前半部分链表头和反转后的后半部分链表头开始遍历
- 比较对应位置的节点值
- 如果有任何不匹配，则不是回文链表

恢复链表（可选）：

- 代码中已注释，如果保持原链表结构不变，可以取消注释
- 再次反转后半部分链表，将其还原为原来的结构

7. 复杂度分析

时间复杂度： $O(n)$

- 找中点需要 $O(n/2)$ 时间
- 反转后半部分需要 $O(n/2)$ 时间
- 比较两部分需要 $O(n/2)$ 时间
- 恢复结构（如果需要）需要 $O(n/2)$ 时间
- 总体时间复杂度为 $O(n)$

空间复杂度： $O(1)$

- 只使用了少量额外变量（指针），不随输入规模变化
- 没有使用额外的数组或栈存储链表元素

8. 如何在面试中讲解

1. 清晰表述问题：

- "这道题要求判断一个链表是否是回文链表，也就是从前向后读和从后向前读是一样的。"

2. 讨论简单但不够优化的方法：

- "最直观的方法是将链表值复制到数组中，然后用双指针判断数组是否回文，但这需要 $O(n)$ 额外空间。"

3. 引入优化方法：

- "为了实现 $O(1)$ 空间复杂度，我们可以将问题分解为几个子问题：找到链表中点、反转后半部分、比较两半部分。"

4. 详细解释算法步骤：

- "首先，我们用快慢指针找到链表中点。"
- "然后，反转从中点开始的后半部分链表。这里我们可以复用LeetCode 206中的链表反转方法。"
- "接着，从头节点和反转后的后半部分的头节点同时开始遍历比较。"
- "最后，如果需保持原链表结构不变，我们可以再次反转后半部分，将链表恢复原状。"

5. 强调算法效率：

- "这个方法的时间复杂度是 $O(n)$ ，空间复杂度是 $O(1)$ ，满足题目的要求。"
- "它利用了我们之前学过的反转链表技巧，将其应用到新的问题中。"

通过这种方法，我们不仅高效地解决了问题，还展示了如何将已有算法知识应用到新的问题上。

找到具有 1 个许可证类型的类似代码